Evaluación práctica: Desarrollo de un sistema de microservicios para la gestión de productos de un minimarket

1. Enunciado

Desarrolle un sistema basado en una arquitectura de microservicios para la gestión de productos de un minimarket. La solución deberá estar compuesta por un microservicio backend que exponga una API REST para administrar los productos, un microservicio frontend desarrollado en PHP que consuma dicha API y un contenedor Docker para ejecutar el sistema gestor de bases de datos MySQL.

El sistema deberá permitir consultar, registrar, editar y eliminar lógicamente productos mediante una interfaz web. Se busca evaluar la capacidad de desarrollar servicios independientes, establecer la comunicación entre ellos, persistir información en una base de datos relacional y configurar el entorno de ejecución mediante contenedores.

2. Requerimientos técnicos

2.1. Microservicio backend: API REST

Implemente un servicio REST utilizando una de las siguientes tecnologías: Quarkus, Spring Boot, PHP o ASP.NET.

El backend deberá conectarse a la base de datos MySQL `db_minimarket_pro413` y exponer los endpoints necesarios para realizar las siguientes operaciones:

- **Consultar:** recuperar todos los productos registrados.
- **Registrar:** crear un producto nuevo.
- **Editar:** actualizar la información de un producto existente.
- **Eliminar lógicamente:** cambiar el estado del producto para que deje de aparecer en el listado habitual, sin borrar físicamente el registro de la base de datos.

Se deberá implementar el manejo adecuado de las solicitudes HTTP, las respuestas de la API y los errores básicos de validación o persistencia.

2.2. Base de datos MySQL en un contenedor

Configure un contenedor Docker para ejecutar MySQL. La base de datos deberá llamarse `db_minimarket_pro413` y contener una tabla llamada `Producto`, con la siguiente estructura obligatoria:

Campo	Tipo de dato MySQL	Restricciones
`id`	`INT`	Clave primaria, autoincremental
`nombre`	`VARCHAR(100)`	No nulo
`proveedor`	`VARCHAR(100)`	No nulo
`fecha_vencimiento`	`DATE`	No nulo
`precio`	`DECIMAL(10,2)`	No nulo, valor mayor o igual a cero
`stock`	`INT`	No nulo, valor mayor o igual a cero
`eliminado`	`TINYINT(1)`	No nulo, valor predeterminado `0`

El campo `eliminado` se utilizará para implementar la eliminación lógica: `0` representa un producto activo y `1` representa un producto eliminado lógicamente.

La configuración deberá incluir un archivo `docker-compose.yml` que permita crear y ejecutar el contenedor de MySQL, definir las credenciales y configurar la persistencia de los datos mediante un volumen.

Importante: la tabla `Producto` deberá conservar los nombres de campos y tipos de datos establecidos para facilitar la verificación directa desde MySQL.

2.3. Microservicio frontend en PHP

Desarrolle una aplicación web en PHP que se comunique con el backend exclusivamente mediante los endpoints de la API REST para las operaciones de gestión de productos.

La interfaz deberá incluir las siguientes funcionalidades:

1. **Listar productos:** mostrar una tabla con los registros activos recuperados desde el backend.
2. **Crear producto:** presentar un formulario para registrar un producto nuevo.
3. **Editar producto:** permitir seleccionar un registro y modificar sus datos.
4. **Eliminar lógicamente:** permitir seleccionar un producto y ejecutar su eliminación lógica mediante el backend.

La tabla deberá mostrar, como mínimo, los campos `id`, `nombre`, `proveedor`, `fecha_vencimiento`, `precio` y `stock`, además de las acciones disponibles.

El frontend no deberá conectarse directamente a MySQL ni ejecutar consultas SQL. Toda operación de lectura o modificación deberá realizarse mediante solicitudes HTTP al backend.

2.4. Comunicación y puesta en marcha

Configure los servicios para que puedan ejecutarse en el entorno definido para la práctica. Documente las direcciones, puertos, dependencias, variables de entorno y comandos necesarios para compilar o preparar cada proyecto e iniciar los servicios.

La solución deberá demostrar la comunicación funcional entre frontend, backend y base de datos. No es obligatorio que los tres componentes estén contenidos en un único archivo Docker Compose: el contenedor de MySQL sí deberá configurarse mediante dicho archivo, mientras que la puesta en marcha de los proyectos de backend y frontend deberá quedar documentada.

3. Entregables obligatorios

A. Informe técnico en PDF

El documento deberá incluir:

- Descripción de la arquitectura y responsabilidades de cada componente.
- Diagrama de arquitectura que represente la comunicación frontend -> API REST -> MySQL.
- Estructura principal de los proyectos y explicación de sus componentes relevantes.
- Configuraciones necesarias para la conexión entre los servicios.
- Configuración de MySQL y explicación del archivo `docker-compose.yml`.
- Comandos de compilación, preparación y puesta en marcha, según la tecnología elegida.
- Evidencias mediante capturas de pantalla de las pruebas de ejecución: listado, creación, edición y eliminación lógica.
- Evidencia de la conservación del registro eliminado en MySQL y de su exclusión del listado de productos activos.

B. Repositorio de código

Entregue el enlace a un repositorio accesible que contenga:

- Proyecto completo del backend.
- Proyecto completo del frontend en PHP.
- Archivo `docker-compose.yml`.
- Archivos de configuración y scripts SQL necesarios, cuando corresponda.
- Instrucciones de ejecución en un archivo `README.md`.

No incluya contraseñas reales, credenciales personales ni otros datos sensibles en el repositorio. Utilice variables de entorno o mecanismos equivalentes para gestionar la configuración.

4. Consideraciones de evaluación

- El sistema debe ser funcional y permitir verificar las cuatro operaciones solicitadas.
- Los nombres de la base de datos, la tabla y los campos deberán respetarse exactamente.
- La eliminación deberá ser lógica, no física.
- El frontend deberá comunicarse con el backend mediante HTTP y no acceder directamente a la base de datos.
- Las evidencias del informe deberán corresponder al sistema implementado.
- El repositorio deberá permitir identificar los proyectos y comprender cómo ejecutar la solución.

Rúbrica de evaluación (10 puntos)

Criterio	Puntaje máximo
Backend REST: implementación funcional de consulta, creación, edición y eliminación lógica de productos.	3
Base de datos MySQL y Docker Compose: estructura de la tabla `Producto`, tipos de datos, restricciones, ejecución del contenedor y persistencia.	2
Frontend en PHP: tabla de productos activos e interfaz funcional para crear, editar y eliminar lógicamente mediante la API REST.	2
Comunicación e integración: comunicación HTTP entre frontend y backend, y persistencia de las operaciones en MySQL.	1
Informe técnico: descripción de la arquitectura, configuraciones, comandos de puesta en marcha y evidencias de las cuatro operaciones.	1
Repositorio: proyectos completos, archivo `docker-compose.yml`, configuraciones necesarias y `README.md` con instrucciones de ejecución.	1
Total	**10**